Traffic Sign Classification with Convolutional Neural Network

Abstract

This project classifies traffic signs from the German Traffic Sign Recognition Benchmark (GTSRB) into 43 classes. The work includes data exploration, preprocessing, CNN architecture comparison, training strategy, hyperparameter tuning, and error analysis. The strongest model is a VGG-style CNN trained on normalized 32×32 RGB images with data augmentation, batch normalization, dropout, RMSprop and learning-rate decay. It achieved 98.41% test accuracy, a macro F1-score of 0.9674 and a weighted F1-score of 0.9838.

1 Problem Formulation

The task is supervised multi-class image classification. Each input is a cropped RGB traffic sign image and the target is one of 43 GTSRB classes. This problem is relevant for driver assistance and autonomous driving because signs encode speed limits, priority rules, warnings and restrictions that affect driving decisions [11]. The output is a single class label, so the model must learn the visual features that distinguish signs with similar colours, borders, and symbols. Some differences are large, for example warning signs versus speed-limit signs, but others are small, such as different speed-limit digits.

The formulation matters because it determines the correct model family. Models such as YOLO predict both bounding boxes and classes in full images [7], but given the fact that the dataset already has cropped images of traffic signs, the task is not an object detection problem. A CNN classifier is therefore the correct method for this assignment, while a detector would only be needed for a different project using full road-scene images.

The project objective was not only to obtain a high accuracy, but also to build a reproducible deep learning experiment. For that reason the notebook records the dataset path, image size, split sizes, seed, training settings, model parameters, and final metrics. The report uses class-wise metrics and error analysis because a traffic sign classifier can look strong in global accuracy while still failing on small or visually similar classes.

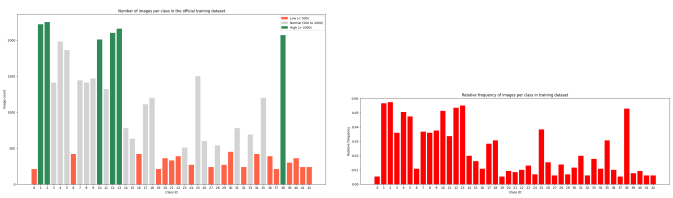
Formally, each image is represented as $x \in R^{32\times32\times3}$, and the corresponding label as $y \in \{1, \ldots, 43\}$. The model outputs class probabilities using a softmax function $p(y = k \mid x) = \frac{\exp(z_k)}{\sum_j \exp(z_j)}$, where z_k denotes the logit (unnormalized score) for class k. The model is trained using categorical cross-entropy loss.

2 Data and Preprocessing

The dataset used is the official GTSRB files: 39,209 rows in `Train.csv`, 12,630 rows in `Test.csv`. The split was taken from the kaggle competition. The test set was kept for final evaluation, while the training set was split into training (80%) and validation (20%). This follows the standard train/validation/test pipeline: training data is used to fit parameters, validation data guides model decisions and hyperparameter tuning and test data estimates final generalization.

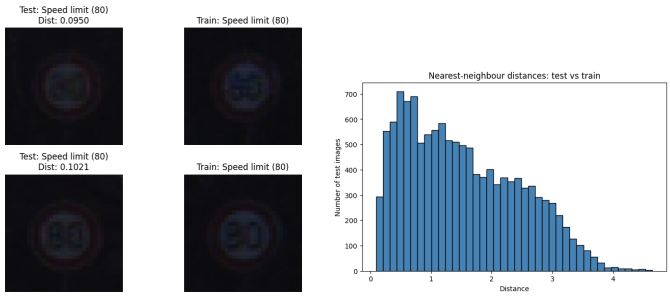
These datasets include resized images of 32×32 and kept as three-channel RGB tensors, which means all the images had the size of $32 \times 32 \times 3$. This resolution is small enough to train quickly locally, while still keeping the colour, shape, and symbol information needed for GTSRB. Pixel values were divided by 255 so that the inputs were in $[0, 1]$, which improves numerical stability for gradient-based learning [2]. Labels were one-hot encoded because the network uses a softmax output and categorical cross-entropy. A fixed random seed (=1) was used for Python, NumPy and TensorFlow before model creation, making the comparisons reproducible.

Exploration showed that the classes are imbalanced: some have 210–450 training images, while several speed-limit and priority classes exceed 2,000. The imbalance is shown in Figure 1 A (class counts) and B (relative frequencies). Some classes represent $< 1\%$ of the dataset. This affects evaluation, as frequent classes can dominate accuracy and hide poor performance on minority classes. For this reason, the final comparison reports macro and weighted F1, not only accuracy. Pixel values range from 0 to 255, motivating normalization to [0, 1]. Brightness also varies strongly, from a mean of 6.64 in the darkest images to 247.49 in the brightest. Many samples are affected by shadows, exposure, or low contrast. Instead of removing these realistic cases, the training data was augmented with random brightness and contrast variations up to 15% to improve robustness and reduce overfitting [8].



(a) Images per class. **(b)** Relative frequencies.
Figure 1: Class imbalance in the GTSRB training set.

Data-quality checks included missing values and duplicate groups, none of which were found. A hash-based leakage check identified 5 train/test and 3 validation/test exact overlaps after preprocessing. These were removed from the train and validation sets only, leaving the test set unchanged, as even small overlaps can bias evaluation. Final splits were 31,362 training images, 7,839 validation images, and 12,630 test images. A nearest-neighbour check was used as a sanity test for near-duplicates [1]. The distance distribution (Figure 2) suggested a tentative cutoff of < 0.1, which flagged one pair; it was retained as it was not an exact duplicate.



(a) Closest pairs. **(b)** NN distances.
Figure 2: Train/test similarity checks.

3 Model Design Choices

CNNs are appropriate for this task because traffic signs are defined by local visual structures such as edges, colours, shapes and symbols. Convolutional layers exploit spatial locality and parameter sharing, allowing hierarchical feature extraction from simple edges to complex sign-level patterns [5, 2]. Early layers can detect simple edges and colour transitions, while later layers can combine them into sign-level features such as a red circular border, a triangular warning shape, or an internal symbol. All models were trained under identical conditions to ensure fair comparison.

The initial experiments focused on a high-capacity AlexNet-style architecture adapted to 32×32 inputs. Although it demonstrated that the task is learnable, its performance was limited by an overparameterized design, with most parameters concentrated in large fully connected layers (37.4M total). This led to unstable loss behaviour and a tendency to overfit, indicating that dense layers dominated learning rather than convolutional feature extraction. Adding data augmentation improved validation accuracy (95.14% to 96.16%), but did not resolve the underlying inefficiency of the architecture.

Subsequent models progressively reduced this imbalance between convolutional representation and classifier size. As shown in Table 1, replacing the flatten-based classifier with global average pooling (`M2c`) significantly reduced parameters while maintaining performance, confirming that much of the dense capacity was redundant. Further reductions in convolutional filters (`M2cl`, `M2cll`) demonstrated that accuracy degrades only slightly when capacity is reduced, suggesting that the task does not require large models.

Table 1: Summary of CNN architecture and optimizer experiments.

Model	Change	Params	Train.	Mem.	Opt.	TrAcc	ValAcc	Best	TrLoss	ValLoss
M2	Flat	1.72M	1.72M	6.57M	Adam	98.15	99.63	99.63	.1595	.0317
M2c	GAP	1.21M	1.21M	4.63M	Adam	98.55	99.64	99.82	.0720	.0099
M2cl	128f	605K	605K	2.31M	Adam	98.47	99.81	99.81	.0761	.0059
M2cll	Half	161K	160K	627K	Adam	97.45	99.63	99.63	.1069	.0119
H	He	161K	160K	627K	Adam	96.88	99.45	99.69	.1180	.0152
H2	AdamW	161K	160K	627K	AdamW	96.99	99.64	99.67	.1198	.0136
H2l	Low LR	161K	160K	627K	AdamW	9.40	15.51	15.51	3.6909	3.1999
SGD	SGD	161K	160K	627K	SGD	95.55	99.49	99.49	.1664	.0189
Reg	L2	605K	605K	2.31M	Adam	98.26	99.77	99.77	.2052	.1274
LR	LR decay	605K	605K	2.31M	Adam	98.94	99.89	99.91	.0498	.0047
RMS	RMSprop	605K	605K	2.31M	RMS	99.37	**99.95**	**99.95**	.0306	**.0024**
Res	Res.	617K	616K	2.35M	RMS	98.79	99.55	99.90	.0444	.0175
Inc	Inc.	104K	104K	407K	RMS	97.24	99.55	99.55	.1012	.0143

The results also highlight the impact of optimization choices. While architectural modifications alone produced modest gains, changing the optimizer and training dynamics led to the largest improvements. In particular, `Model 2cl RMS` achieved the best validation accuracy (99.95%) without increasing model size, indicating that performance is primarily driven by training strategy rather than capacity. Conversely, suboptimal hyperparameter choices (e.g. low learning rate in `H2l`) resulted in severe underfitting, reinforcing the sensitivity of the model to optimization settings.

More complex architectural variants, such as residual connections and Inception-style modules, did not yield further improvements despite increased design complexity. This suggests diminishing returns from architectural scaling in this setting, where the input resolution is low and the task is dominated by local patterns.

The final `Model 2cl RMS` architecture has four convolutional stages, each followed by max pooling, batch normalization and dropout, then

global average pooling, one 128-unit dense layer and a 43-class softmax output. Batch normalization stabilizes activations during training [4], while dropout reduces co-adaptation and overfitting [10]. Global average pooling keeps the classifier small by summarizing feature maps before the dense layer. This addresses the main weakness of the earlier models, where the classifier head was too large for 32×32 input images.

4 Training Strategy and Tuning

All models were trained for 20 epochs with a batch size of 64, using categorical cross-entropy because the task is single-label multi-class classification. The first VGG-style models used Adam as a strong adaptive baseline optimizer [2]. After the AlexNet-style and dense-head models proved inefficient, the tuning focused first on reducing capacity: replacing the flatten-based classifier with global average pooling, reducing the number of filters and testing a smaller lightweight version.

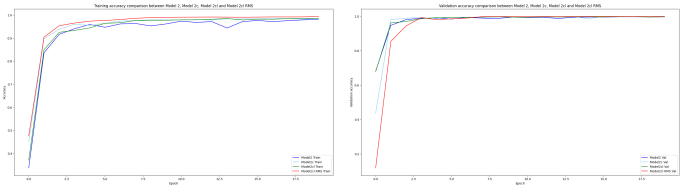
After the architecture had been simplified, the next step was to test whether training could be improved without making the model larger. For this reason, I compared He initialization, AdamW with weight decay, L2 regularization, SGD with momentum, RMSprop and ReduceLROnPlateau. Two functional exploratory variants were also tested: a ResNet-style model with residual skip connections [3] and an Inception-style model (GoogLeNet) with multi-branch convolutions [12]. The best result came from `Model 2cl RMS`, which kept the `Model 2cl` architecture but changed the optimizer to RMSprop with learning-rate decay [13].

Augmentation was applied only during training, so validation and test results were measured on unchanged images. This improved robustness during fitting while keeping evaluation realistic. Final model selection was checked using test accuracy and F1-scores, since validation accuracy alone can hide class-level weaknesses.

5 Performance Analysis

Table 2 shows that `Model 2cl RMS` achieved the strongest overall performance, with 98.74% test accuracy, macro F1 of 0.9780 and weighted F1 of 0.9872. The comparison is especially informative because the larger or more complex variants, such as ResNet-style and Inception-style models, did not improve test performance. This suggests that, for this cropped and low-resolution GTSRB classification task, better optimization and regularization were more beneficial than increasing architectural complexity. Since the dataset is imbalanced, macro F1 is the most relevant complementary metric because it weights all classes equally, while weighted F1 reflects the class distribution and can be dominated by frequent classes [9, 6]. The small gap between macro and weighted F1 indicates that the final model performs consistently across both frequent and minority classes.

The training curves in Figure 3 show that the models converged quickly within the first few epochs. Validation accuracy remained high and stable, indicating no clear overfitting. Importantly, the best performance was not obtained by increasing model size.



(a) Training acc. **(b)** Validation acc.

Figure 3: Convergence across selected models.

`Model 2cll` highlights the efficiency–accuracy trade-off, achieving 98.08% test accuracy with only 160,507 parameters (26.5% of `Model 2cl`). This suggests that a large proportion of the performance can be retained with substantially reduced capacity. In contrast, more complex variants such as ResNet and Inception do not yield measurable gains, indicating diminishing returns from architectural complexity in this setting. Overall, the results point to an efficiency plateau, where improvements are limited by the task rather than model capacity.

Table 2: Combined performance comparison of the main models.

Model	Test	Train	Val.	Macro F1	Weighted F1
Model 2	97.30	98.15	99.63	0.9568	0.9725
Model 2cl	98.26	99.19	99.90	0.9696	0.9822
Model 2cll	98.08	98.20	99.80	0.9651	0.9806
Model 2cl RMS	**98.74**	**99.47**	**99.96**	**0.9780**	**0.9872**
RMS ResNet	97.34	98.79	99.55	0.9615	0.9734
RMS Inception	97.24	97.24	99.55	0.9513	0.9722

6 Error Analysis and Limitations

The confusion matrices in Figure 4 are diagonal, especially for `Model 2cl RMS`, which confirms the high overall accuracy. The remaining errors are concentrated in visually similar classes. For the best model, the most frequent mistake was *Speed limit (120)* predicted as *Speed limit (80)* 12 times. Other repeated errors included *End speed limit(80)* and *Bumpy road*. The wrong-prediction tables in Tables 3 and 4 show both the most frequent confusion pairs and the classes that remain hardest to classify across models.

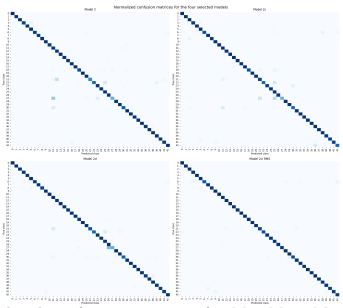


Figure 4: Normalized confusion matrices for M2, M2c, M2cl, and M2cl RMS.

Table 3: Final M2cl RMS errors.

Actual	Pred.	N
Speed 120	Speed 80	12
Ice/snow	Bicycles	9
End speed 80	EnP >3.5t	9
End speed 80	Speed 60	8
Bumpy road	Road work	6
Priority road	No veh.	6
Speed 80	Speed 60	5
Bumpy road	Speed 20	5
EnP >3.5t	End no pass	5
Gen. caution	Traffic sig.	4

Table 4: Hardest actual classes.

Class	M2	M2c	M2cl	RMS	Tot.
Gen. caution	34	57	15	6	112
Ice/snow	25	32	27	16	100
Bumpy road	29	26	24	14	93
Pedestrians	30	30	30	1	91
Speed 60	31	24	17	5	77
Speed 120	2	24	32	14	72
End speed 80	8	21	8	24	61
Double curve	31	7	13	7	58
Speed 70	20	8	10	4	42
Priority road	13	9	9	6	37

The class-level analysis Figure 5 shows that most classes have high precision, recall, and F1, but a few remain weaker. In `Model 2cl RMS`, *Pedestrians* improved compared with the earlier models, reaching an F1-score of 0.97. However, *Double curve* (F1 = 0.83), *Roundabout* (F1 = 0.89) and *End speed limit(80)* (F1 = 0.91) remained harder to classify. These errors are reasonable because some traffic signs share the same shape and differ only by small internal symbols.

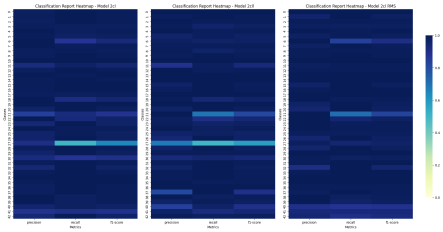


Figure 5: Classification report heatmaps for M2cl, M2cll, and M2cl RMS.

The main limitation is that this project classifies already cropped traffic signs. It does not detect signs in full road scenes, so it is not a complete autonomous-driving perception system. A real deployment would also need object detection, tracking and robustness to motion blur, weather variation, camera angle changes, occlusion, night-time images and country-specific sign distributions. Another limitation is the fixed 32×32 input size: it makes training efficient, but it can remove small details such as digits, arrows, or internal symbols, which probably contributed to errors between visually similar signs. Class imbalance also remains a challenge, especially for minority classes. Finally, although the nearest-neighbour distance check helped identify exact and near-duplicate risks, it was only a simple image-similarity sanity check rather than a full guarantee that the splits are completely independent. Since several final models were also compared on the test set, the reported accuracy should be read as the best result from these experiments, not as a single untouched test evaluation.

Future work could address these limitations by moving from cropped classification to full-scene traffic-sign detection, for example by combining a detector (YOLO) with the final CNN classifier. The model could also be tested at higher input resolutions to preserve small digits and symbols and with stronger augmentation for blur, rain, occlusion, night-time lighting and perspective changes. Finally, repeated stratified splits or cross-validation would make the model comparison more robust, while targeted augmentation or class weighting could improve performance on minority and visually similar classes.

Consequently, Table 5 shows that strong performance did not require a large network. The final compact VGG-style CNN with GAP used only 457k parameters and achieved .984 test accuracy, .967 macro F1, and .984 weighted F1. This confirms that careful optimization and regularization, rather than architectural complexity, were the key drivers of performance.

Table 5: Final model summary.

Item	Value
Dataset/Input	GTSRB, 32×32 RGB
Model	VGG + Global Average Pooling
Opt./Reg.	RMSprop, Batch normalization + dropout + aug.
Params	457,547
Acc./Macro/Weighted F1	.9841 / .9674 / .9838

References

[1] Björn Barz and Joachim Denzler. Do we train on test data? purging cifar of near-duplicates. *Journal of Imaging*, 6(6):41, 2020. doi: 10.3390/jimaging6060041.

[2] Ian Goodfellow, Yoshua Bengio, and Aaron Courville. *Deep Learning*. MIT Press, 2016. URL `https://www.deeplearningbook.org/`.

[3] Kaiming He, Xiangyu Zhang, Shaoqing Ren, and Jian Sun. Deep residual learning for image recognition. In *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, pages 770–778, 2016. doi: 10.1109/CVPR.2016.90. URL `https://www.cv-foundation.org/openaccess/content_cvpr_2016/html/He_Deep_Residual_Learning_CVPR_2016_paper.html`.

[4] Sergey Ioffe and Christian Szegedy. Batch normalization: Accelerating deep network training by reducing internal covariate shift. In *Proceedings of the 32nd International Conference on Machine Learning*, pages 448–456, 2015. URL `https://proceedings.mlr.press/v37/ioffe15.html`.

[5] Yann LeCun, Léon Bottou, Yoshua Bengio, and Patrick Haffner. Gradient-based learning applied to document recognition. *Proceedings of the IEEE*, 86(11):2278–2324, 1998. doi: 10.1109/5.726791.

[6] David M. W. Powers. Evaluation: From precision, recall and f-measure to roc, informedness, markedness and correlation. *Journal of Machine Learning Technologies*, 2(1):37–63, 2011.

[7] Joseph Redmon, Santosh Divvala, Ross Girshick, and Ali Farhadi. You only look once: Unified, real-time object detection. In *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, pages 779–788, 2016. doi: 10.1109/CVPR.2016.91. URL `https://www.cv-foundation.org/openaccess/content_cvpr_2016/html/Redmon_You_Only_Look_CVPR_2016_paper.html`.

[8] Connor Shorten and Taghi M. Khoshgoftaar. A survey on image data augmentation for deep learning. *Journal of Big Data*, 6(60), 2019. doi: 10.1186/s40537-019-0197-0.

[9] Marina Sokolova and Guy Lapalme. A systematic analysis of performance measures for classification tasks. *Information Processing & Management*, 45(4):427–437, 2009. doi: 10.1016/j.ipm.2009.03.002.

[10] Nitish Srivastava, Geoffrey Hinton, Alex Krizhevsky, Ilya Sutskever, and Ruslan Salakhutdinov. Dropout: A simple way to prevent neural networks from overfitting. *Journal of Machine Learning Research*, 15(56):1929–1958, 2014. URL `https://jmlr.org/papers/v15/srivastava14a.html`.

[11] Johannes Stallkamp, Marc Schlipsing, Jan Salmen, and Christian Igel. The german traffic sign recognition benchmark: A multi-class classification competition. In *Proceedings of the International Joint Conference on Neural Networks (IJCNN)*, pages 1453–1460, 2011. doi: 10.1109/IJCNN.2011.6033395. URL `https://www.ini.rub.de/upload/file/1470692848_f03494010c16c36bab9e/StallkampEtAl_GTSRB_IJCNN2011.pdf`.

[12] Christian Szegedy, Wei Liu, Yangqing Jia, Pierre Sermanet, Scott Reed, Dragomir Anguelov, Dumitru Erhan, Vincent Vanhoucke, and Andrew Rabinovich. Going deeper with convolutions. In *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, pages 1–9, 2015. doi: 10.1109/CVPR.2015.7298594. URL `https://www.cv-foundation.org/openaccess/content_cvpr_2015/html/Szegedy_Going_Deeper_With_2015_CVPR_paper.html`.

[13] Tijmen Tieleman and Geoffrey Hinton. Lecture 6.5—rmsprop: Divide the gradient by a running average of its recent magnitude. COURSERA: Neural Networks for Machine Learning, 2012.